

Conflict-Based Search for Optimal Multi-Agent Path Finding

A Reproduction and Extension Study

Nimrod Netzer | Elad Damti | Kfir Dahan

Search in Artificial Intelligence, Bar-Ilan University, 2026

Abstract

Multi-Agent Path Finding (MAPF) is the problem of planning collision-free paths for k agents on a shared graph while minimizing total travel cost. Solving MAPF optimally is NP-hard, and joint-state-space A* fails due to exponential state-space growth. We reproduce key results from Sharon et al. (2015), who introduced Conflict-Based Search (CBS), a two-level algorithm separating high-level conflict resolution from low-level single-agent planning. We implement CBS independently in Python, reproduce two central results (success rate vs. agent count and runtime vs. Joint-State-Space A*), and contribute an extension comparing open grids against warehouse maps with bottlenecks. CBS success drops from 100% at 4 agents to 60% at 20 agents while Joint A* fails completely from 8 agents onward. Warehouse maps reduce CBS success further, from 64% to 28% at 20 agents, with 3.7x more CT node expansions.

1. Introduction

MAPF arises in automated warehouses, airport traffic, and railway scheduling [Sharon et al., 2015]. The naive approach of joint-state-space A* is infeasible: state space grows as $O(|V|^k)$, reaching ~ 9.7 million states for $k=10$ agents on $|V|=5$ vertices. Our experiments confirm this: Joint A* succeeds on only 32% of 6-agent instances and 0% of 8-agent instances within 5 seconds. CBS solves this via a Constraint Tree (CT) that branches on conflicts and a low-level Time-Space A* (TSA*) that plans paths per agent subject to constraints. We reproduce two results from Sharon et al. (2015) and study how map topology affects CBS performance.

2. Selected Paper Description

2.1 Problem Formulation

MAPF is defined on graph $G=(V,E)$ with k agents, each with start s_i and goal g_i . Agents move or wait each timestep. A valid solution has no vertex conflicts (same vertex, same time) or edge conflicts (agents swap positions). Objective: minimize SOC = sum of $|p_i|$ for all agents.

2.2 Conflict Types

Sharon et al. (2015) identify five conflict types: (1) Vertex: same vertex, same time. (2) Edge: agents swap positions. (3) Following: one agent follows another on the same edge. (4) Cycle: cyclic dependency. (5) Swapping: position exchange. We detect and resolve vertex and edge conflicts, sufficient for correctness and optimality.

2.3 Why Joint A* Fails

The joint state space is $O(|V|^k)$. For $k=30$ agents on a realistic map, exhaustive joint search is infeasible. CBS avoids this by planning agents independently and resolving only actual conflicts.

2.4 CBS Algorithm

Low level - TSA*: State = (vertex v , timestep t). Vertex constraint (v,t) forbids occupying v at t . Edge constraint forbids a directional move at t . Heuristic = Manhattan distance (admissible on 4-connected grids). TSA* checks future constraints at the goal before terminating.

High level - CT: Binary tree; each node stores constraints per agent, TSA* paths, and SOC. CBS: (1) Root: plan each agent independently. (2) Find first conflict. (3) If none, return optimal solution.

(4) Branch: left child constrains agent i, right child constrains agent j. (5) Replan constrained agent. (6) Push both children to min-heap by SOC. Repeat. CBS is complete and optimal [Sharon et al., 2015].

2.5 CBS Improvements

The paper introduces: Prioritizing Conflicts (cardinal conflicts first), CBS with Heuristics (high-level admissible heuristic), Disjoint Splitting (positive and negative constraints), and Conflict Reasoning. We implement basic CBS only.

3. Project Description

3.1 Implementation

We implemented CBS independently in Python 3.13 without the original authors' code, across seven modules: **graph.py** (4-connected Grid with obstacles); **tsa_star.py** (TSA* with constraint checking and future-constraint-aware goal detection); **conflict.py** (vertex and edge conflict detection); **cbs.py** (CT, CTNode structure, CBS main loop, min-heap by SOC); **joint_astar.py** (true Joint-State-Space A* baseline matching the paper's intended comparison); **benchmark.py** (map generators, instance generator, experiment runner); **visualize.py** (matplotlib plots).

3.2 Team Contributions

Nimrod Netzer: tsa_star.py (TSA*, constraint checking, goal detection). Abstract, Sections 1-2.

Elad Damti: conflict.py and cbs.py (CTNode, CBS loop). Sections 3-4.

Kfir Dahan: joint_astar.py, benchmark.py, visualize.py. All experiments. Sections 5-6, Reproducibility, AI Disclosure.

All three members participated in testing, debugging, defense preparation, and the video.

3.3 Key Implementation Choices

Baseline: Joint-State-Space A* (joint_astar.py) searches all agent positions simultaneously, matching the paper's intended comparison. Unlike independent A*, it guarantees conflict-free solutions.

Goal model: Agents stay at goal indefinitely; paths padded for conflict checking.

Conflict selection: First conflict found by timestep then agent pair index.

Seeds: Instance seed = 42 + n_agents x 1000 + instance_index. Deterministic.

4. Experiments

4.1 Experimental Setup

Hardware: 12th Gen Intel Core i7-1255U, 10 cores, 1.70 GHz, 16 GB RAM, Windows 11 Home.

Software: Python 3.13; matplotlib 3.11; standard library only (heapq, collections, csv, json, time). No external MAPF libraries.

Benchmark instances: 20x20 grid maps generated randomly. Open grid: ~10% obstacle density (seed=1). Warehouse grid: alternating shelf rows with single-cell corridors every 4 columns. 25 random instances per agent count.

Algorithms: (1) CBS as described in Section 3. (2) Joint-State-Space A* as the paper's baseline.

Parameter settings: CBS: first-found conflict, stay-at-goal model, max_t=200. Joint A*: admissible heuristic = sum of Manhattan distances.

Time limits: 5s for Joint A* (reproduction); 30s for CBS (reproduction); 15s for CBS (extension). Exceeded instances counted as failures.

Memory limits: No explicit cap; bounded by OS (16 GB).

Number of runs: 1 per instance (deterministic algorithm).

Random seeds: Instance seed = 42 + n_agents x 1000 + instance_index. Map seed = 1.

Differences from paper: 20x20 grid (smaller than paper benchmarks); Python instead of C++ (~10-50x slower); shorter time limits. These explain earlier success rate drops.

4.2 Reproduced Results

Table 1 shows CBS vs. Joint A* on the open 20x20 grid. Joint A* collapses from 8 agents onward, confirming exponential state-space explosion. CBS scales significantly better but begins timing out beyond 15 agents.

Agents	CBS Success	CBS Time (s)	CT Nodes	SOC	Joint A* Success
4	100%	0.0018	1.7	56.8	100%
6	92%	0.0045	3.7	83.8	32%
8	100%	0.0078	5.4	114.3	0%
10	96%	0.0168	12.7	132.6	0%
12	96%	0.0719	70.6	158.1	0%
15	64%	0.5431	334.3	204.8	0%
18	64%	0.3147	251.3	236.4	0%
20	60%	1.0958	716.3	266.7	0%

Table 1: CBS vs. Joint-State-Space A* on open 20x20 grid. 25 instances per count. Joint A* limit: 5s. CBS limit: 30s. Means over successful instances.

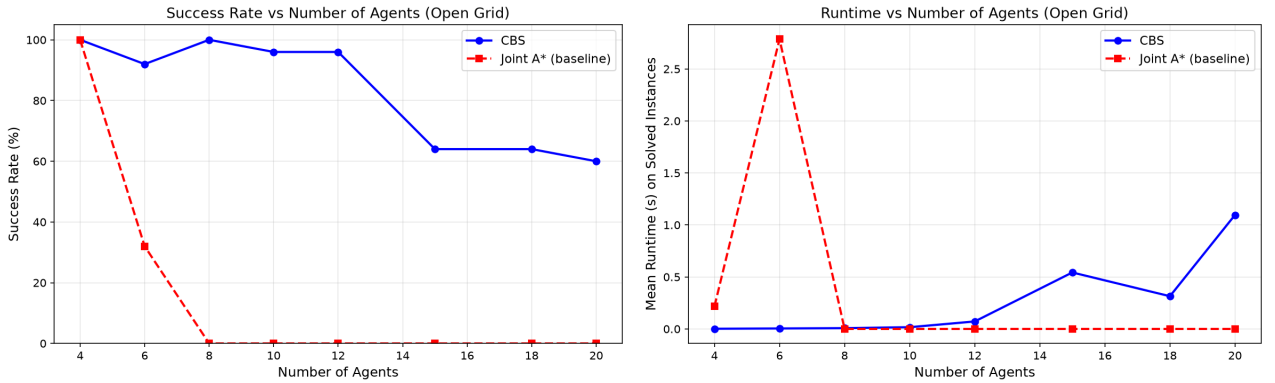


Figure 1 (left): Success rate vs. agents. Figure 2 (right): Mean CBS runtime on solved instances.

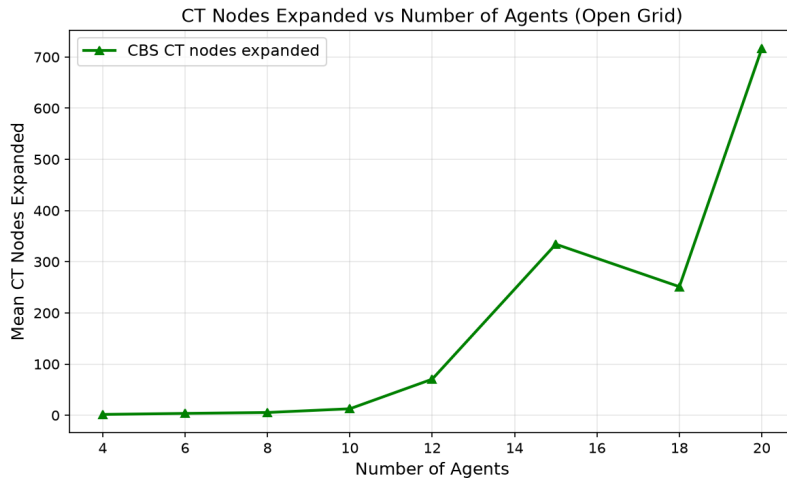


Figure 3: Mean CT nodes expanded vs. number of agents.

CT nodes jump from 70.6 at 12 agents to 334.3 at 15 agents, reflecting CBS's exponential worst-case behavior consistent with Sharon et al. (2015).

4.3 Extension Results - Map Topology Study

Research question: Do warehouse maps with narrow corridors create more conflicts and reduce CBS success compared to open grids?

Agents	Open Success	Open CT Nodes	Open Time (s)	WH Success	WH CT Nodes	WH Time (s)
4	100%	1.7	0.0017	100%	3.9	0.0037
6	92%	3.7	0.0044	100%	3.2	0.0031
8	100%	5.4	0.0071	100%	19.4	0.0148
10	96%	12.7	0.0135	100%	125.9	0.2617
12	100%	131.7	0.3525	96%	633.7	0.8381
15	72%	1047.1	1.6304	72%	655.3	0.5128
18	68%	920.1	0.9647	36%	1845.1	1.6920
20	64%	877.4	1.5390	28%	3258.3	3.1362

Table 2: CBS on Open vs. Warehouse Grid (WH). 25 instances per count. Time limit: 15s.

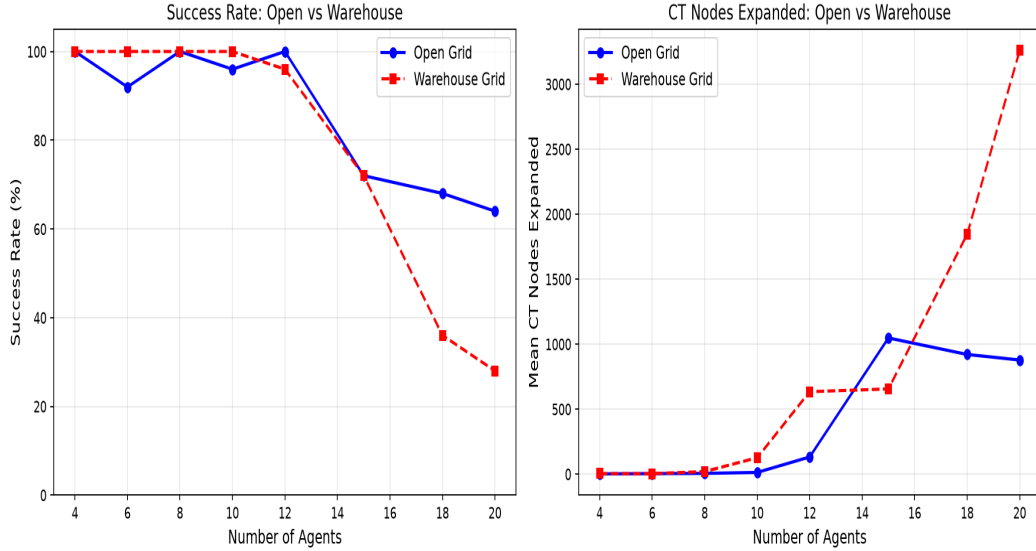


Figure 4: CBS success rate - Open Grid vs. Warehouse Grid.

Warehouse maps dramatically underperform at high agent counts: 20 agents - Open 64% vs. Warehouse 28%. CT nodes at 20 agents: 877 (open) vs. 3,258 (warehouse, 3.7x). Bottlenecks force agents into the same corridor cells, multiplying conflicts and CT branches.

5. Discussion

5.1 Agreement with Original Paper

Results are qualitatively consistent with Sharon et al. (2015): CBS achieves near-perfect success for small agent counts and degrades as agents increase; Joint A* fails completely from 8 agents; CT nodes grow exponentially beyond 12 agents. Quantitative differences are expected due to Python vs. C++, smaller 20x20 grid, and shorter time limits.

5.2 Extension Findings

Warehouse maps confirm that bottleneck structure severely impacts CBS. Agents must share narrow corridor cells, creating vertex conflicts that cascade: each CT branch forces a longer detour, generating new conflicts in adjacent corridors. At 20 agents, CBS expands 3,258 CT nodes on

warehouse maps vs. 877 on open grids (3.7x). This confirms that CBS improvements - conflict prioritization and high-level heuristics - are especially needed in warehouse environments.

5.3 Limitations

Python is ~10-50x slower than C++, causing earlier timeouts. The 20x20 grid is smaller than the paper's benchmarks. We implemented basic CBS only, without the improvements that would extend the solvable agent range.

6. Conclusion

We implemented CBS from scratch in Python and reproduced its two central results. CBS finds provably optimal, collision-free solutions while scaling far better than Joint A*, which fails completely from 8 agents. Our extension shows warehouse bottlenecks reduce CBS success from 64% to 28% at 20 agents, with 3.7x more CT expansions. Future work should apply CBS improvements specifically to warehouse environments where the performance gap is largest.

Reproducibility Statement

Results reproduced: (1) Success rate of CBS vs. Joint-State-Space A* as a function of agent count on grid maps, corresponding to the success rate figures in Sharon et al. (2015). (2) Runtime and CT node expansion statistics on solved instances.

Results not reproduced: Experiments on Moving AI benchmark maps (e.g., Paris_1_256); comparisons to ICTS or other MAPF algorithms; CBS improvement variants (prioritized conflicts, disjoint splitting, high-level heuristics).

Implementation basis: New independent implementation in Python 3.13. The original authors' code was not used at any stage.

Changes from original paper: 20x20 grid vs. larger benchmarks; Python instead of C++; time limits of 5s (Joint A*) and 30s (CBS) vs. the paper's longer limits; randomly generated maps instead of paper benchmark files.

Agreement with original: Qualitative trends match: CBS scales better than joint search, success degrades with agent count, CT nodes grow exponentially beyond ~12 agents. Absolute numbers differ due to language, grid size, and time limits.

Files included: graph.py, tsa_star.py, conflict.py, cbs.py, joint_astar.py, benchmark.py, visualize.py, main.py, generate_report.py; results/reproduce_open.csv, results/extension_open.csv, results/extension_warehouse.csv; results/fig1-fig4.png; results/report.pdf.

How to run:

```
pip install matplotlib
python main.py --mode reproduce --n-instances 25 --time-limit 30 --agent-counts 4 6 8 10 12 15 18 20
python main.py --mode extension --n-instances 25 --time-limit 15 --agent-counts 4 6 8 10 12 15 18 20
```

AI Tools Disclosure

Claude (Anthropic): Used as a technical assistant for code optimization, syntax debugging, and editorial review of the documentation.

All core algorithms (including TSA* and CBS) were conceptualized and implemented directly by the team. The team has thoroughly validated all project components and takes full responsibility for the integrity, originality, and performance of the final output.

References

- Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219, 40-66. <https://doi.org/10.1016/j.artint.2014.11.006>
- Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144-148.